



LISTA DE EXERCÍCIOS 2

GABARITO...

OBS. Resoluções em sala de aula podem englobar mais pontos de vista e atributos do que apenas as respostas simplificadas apresentadas no gabarito.

1. Qual é o propósito de chamadas de sistema (*system calls*)?

Resposta: chamadas de sistema permitem que processos em espaço de usuário requisitem serviços do SO.

2. Quais são as cinco principais atividades de um SO com relação ao gerenciamento de processos?

Resposta: 1) criação e remoção de processos de usuário e sistema, 2) suspensão e continuação de processos, 3) provisão de mecanismos para sincronização de processos, 4) provisão de mecanismos para comunicação de processos, 5) provisão de mecanismos para tratamento de *deadlock*.

3. Quais são as três principais atividades de um SO com relação ao gerenciamento de memória?

Resposta: 1) rastrear que partes da memória estão em uso no momento e por quem, 2) decidir quais processos serão carregados na memória quando o espaço se tornar disponível, 3) alocar e liberar espaço de memória quando necessário.

4. Quais são as três principais atividades de um SO com relação ao gerenciamento de armazenamento secundário?

Resposta: 1) gerenciamento de espaço livre, 2) alocação de espaço em disco, 3) escalonamento de disco.

5. Qual é o propósito do interpretador de comandos? Por que este é geralmente separado do kernel?

Resposta: Ler comandos provenientes do usuário (ou de um arquivo contendo comandos) e os executar, geralmente “transformando-os” em uma ou mais chamadas de sistema. Geralmente, não é parte do *kernel* considerando que o interpretador de comandos é um programa que está sujeito a mudanças e atualizações.

6. Quais chamadas de sistema precisam ser executadas por um interpretador de comandos com o intuito de iniciar um novo processo?

Resposta: Em sistemas Unix, uma chamada de sistema *fork*, seguida por uma chamada de sistema *exec* precisam ser realizadas para iniciar um novo processo. O *fork* clona o processo em execução que a chamou, enquanto o *exec* sobrescreve o processo (filho) chamador baseado em um executável diferente.

7. Qual é o propósito de programas do sistema?

Resposta: Programas de sistema podem ser interpretados como “pacotes” de chamadas de sistema úteis. Eles oferecem funcionalidades básicas para usuários de modo que estes não precisem escrever seus próprios programas para resolver problemas comuns.

8. Qual é a principal vantagem da abordagem em camadas para o projeto de sistema? Quais são as desvantagens de usar a abordagem em camadas?

Resposta: Como em todos os casos de projeto modular, um SO projetado desta forma possui diversas vantagens. O sistema é mais facilmente depurável e modificável, pois mudanças afetam somente seções limitadas do sistema ao invés de afetar todas as seções do SO. A informação é mantida apenas onde é necessária e está acessível somente dentro de uma área restrita e bem definida, de modo que quaisquer *bugs* afetando aqueles dados estão limitados a um módulo ou camada específica. Desvantagens: aumento na complexidade do projeto devido a estrutura modular e possível comunicação entre módulos (ex. via passagem de mensagens) e sobrecarga de desempenho considerando a comunicação entre espaço de usuário e espaço de kernel.

9. Liste cinco serviços oferecidos pelo SO. Explique como cada um deles oferece conveniência aos usuários. Explique também, em quais casos seria possível para programas em espaço de usuário oferecer estes serviços.

Resposta: Execução de programa. O SO carrega os conteúdos (ou seções) de um arquivo na memória e começa sua execução. Um programa em espaço de usuário não é confiável o suficiente para alocar o tempo de CPU de maneira adequada. **Operações de E/S.** Discos, fitas, linhas seriais, e outros dispositivos devem ser acessados em baixo nível. O usuário precisa apenas especificar o dispositivo e a operação a ser desempenhada, enquanto o sistema converte as requisições em comandos específicos de dispositivo ou controlador. Programas de espaço de usuário não são confiáveis para acessar apenas dispositivos que eles têm permissão para acessar e também, para controlar esse acesso apenas para quando o dispositivo estiver ocioso (desocupado). **Manipulação de sistema de arquivo.** Existem muitos detalhes na criação, remoção, alocação e nomeação de arquivos, detalhes estes que os usuários não teriam condições de desempenhar. Blocos de espaço em disco são usados por arquivos e devem ser rastreados. Apagar arquivo requer a remoção do nome do arquivo e a liberação dos blocos alocados. Proteções devem também ser verificadas de modo a assegurar o acesso apropriado ao arquivo. Programas de usuário não seriam confiáveis o suficiente considerando a manutenção da proteção necessária ou a alocação e liberação de blocos nas condições adequadas (quando estes estão livres ou já não possuem mais serventia). **Comunicações.** A passagem de mensagem entre sistemas requer que mensagens sejam transformadas em pacotes de informação, enviadas pela controladora de rede, transmitida por um meio de comunicação e montadas no sistema destino. A ordenação e correção de dados deve acontecer. Novamente, programas de usuário poderiam não coordenar o acesso ao dispositivo de rede, ou ainda, receber pacotes destinados a outros processos. **Deteção de erro.** A detecção de erro ocorre em nível de hardware e software. No nível de hardware, todas as transferências de dados devem ser inspecionadas de modo a assegurar que dados não foram corrompidos em trânsito. Todos os dados na mídia devem ser verificados de modo a assegurar que eles não mudaram desde que foram escritos na mídia. No nível de software, a mídia deve ser verificada por consistência de dados; por exemplo, se o número de blocos alocados e liberados do disco são equivalentes ao número total registrado no dispositivo. Assim, erros são frequentemente independentes de processo (por exemplo, a corrupção de dados em um disco), de modo que deve existir um programa global (o SO) que trata todos os tipos de erros. Além disso, quando o SO trata os erros, os processos não precisam conter códigos para identificar e corrigir todos os erros possíveis no sistema.

10. Quais são as principais vantagens da abordagem de *microkernel* para o projeto de sistema?

Resposta: Os benefícios dessa abordagem incluem (a) a inserção de um novo serviço não requer modificações no kernel, (b) é mais seguro, considerando que mais operações são realizadas em espaço de usuário ao invés de espaço de kernel, e (c) um projeto de kernel mais simples e funcional resulta em um SO mais confiável.

11. Por que alguns sistemas armazenam o SO em *firmware* enquanto outros o fazem em disco?

Resposta: Para certos dispositivos, como PDAs, *handhelds*, *smartphones* etc um disco com um sistema de arquivos pode não estar disponível no dispositivo. Nesta situação, o SO deve ser armazenado no *firmware*.

12. Como poderia um sistema ser projetado para permitir uma escolha de SOs em *boot*? O que o programa de *bootstrap* precisaria fazer?

Resposta: Considere um sistema que precisaria executar o Windows e três outras distribuições de Linux (ex. RedHat, Debian e Ubuntu). Cada SO será armazenado no disco. Durante o processo de inicialização (*system boot-up*), um programa especial (o gerenciador de *boot*) determinará qual SO será carregado. Isso significa que ao invés de carregar um SO logo no início, o gerenciador de *boot* será o programa carregado inicialmente. É esse gerenciador de *boot* que é responsável por determinar qual sistema será carregado. Geralmente, gerenciadores de *boot* devem ser armazenados em certas posições do disco, de modo a serem reconhecidos durante o procedimento de inicialização. Gerenciadores de *boot* frequentemente oferecem ao usuário uma seleção de sistemas a serem carregados. Também são geralmente projetados para carregar um SO padrão caso nenhuma escolha tenha sido selecionada pelo usuário.

13. Discorra sobre as questões envolvidas na implementação de *kernels* monolíticos e *microkernels* discutidas no famoso debate realizado entre A. S. Tanenbaum e Linus Torvalds no início da década de 90.

Resposta: link: <http://www.oreilly.com/openbook/opensources/book/appa.html>